Information Retrieval Indexing

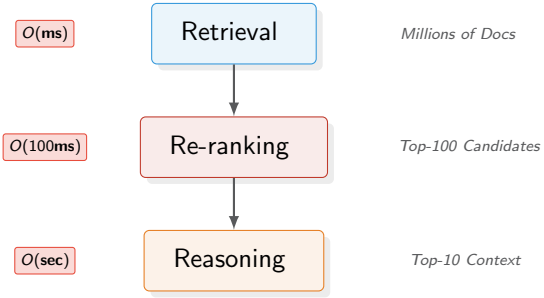
Lecture 2: Sparse and Dense Indexing

Avishek Anand

Information Retrieval

March 2, 2026

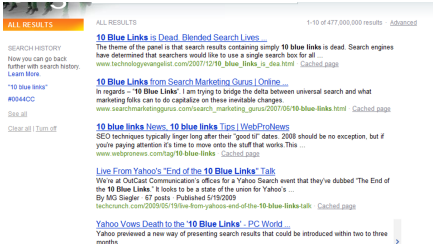
Retrieval → Re-ranking → Reasoning



Key Principle

Indexing sets the ceiling. Everything else (re-ranking, LLM reasoning) can only refine the results that retrieval finds.

Traditional Retrieval: 10 Blue Links

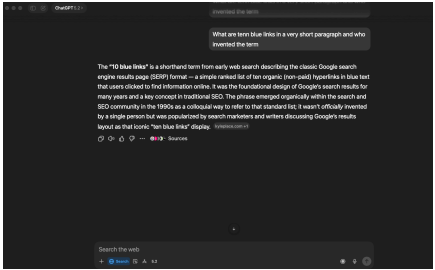


- **User Task:** Search for information.
- **Output:** A list of relevant documents.
- **Interaction:** User clicks links, reads, and synthesizes.
- **Classic System:** Google, Bing, Yahoo in the 2000s.

Problem

User does all the hard work of reading and combining information!

Modern IR: Retrieval-Augmented Generation (RAG)

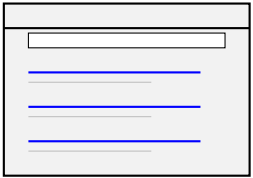


- **User Task:** Ask a question.
- **Output:** A synthesized answer in natural language.
- **Process:**
  1. **Retrieve** relevant facts.
  2. **Augment** LLM prompt with facts.
  3. **Generate** final answer.
- **Modern System:** ChatGPT, Perplexity, Bing Chat.

The Interface Changed. The Engine Did Not.

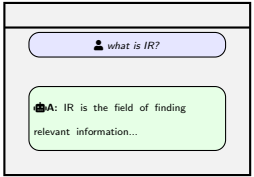
Old Interface (2000s):

- Output: Ranked documents
- Interaction: User reads/clicks



Modern Interface (2024+):

- Output: Generated answers
- Interaction: Model synthesizes facts

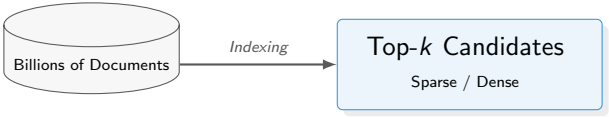


Core Message

Retrieval now serves models instead of users.
But the underlying components (Index, Retrieval, Relevance) remain the same.

Why Do We Need Indexing?

1. **Scale:** Cannot score all documents in real-time.
2. **Latency:** Users (and LLMs) expect sub-second responses.
3. **Feasibility:** Must reduce millions $\rightarrow$ hundreds of candidates.



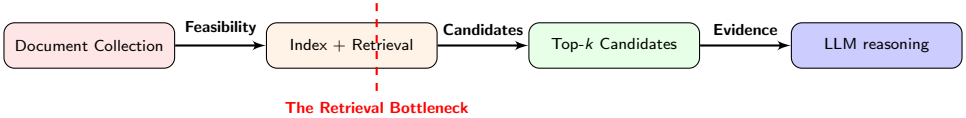
The Bottom Line

Indexing is not just an optimization. It is the bridge to feasibility.

What Does Indexing Control?

Indexing Determines

- **Recall Ceiling:** Which documents can ever be retrieved.
- **Latency Budget:** How many candidates can be scored (fast vs slow).
- **Cost per Query:** Hardware requirements and compute cost.
- **What the LLM is allowed to see:** Retrieval is the LLM's "eyes".



Hard Constraint

If retrieval misses it, the LLM cannot reason about it.

Outline

1. Sparse Indexing: The Classical Approach
2. Dense Vector Indexing for Neural Retrieval
3. Design Decisions: When to Use What
4. Summary and Next Steps

The Inverted Index: Still Relevant in 2024

What Powers These Systems?

- Google Search
- Elasticsearch / OpenSearch
- Database full-text search (PostgreSQL, MySQL)
- RAG sparse retrieval component

Answer: **Inverted indexes**

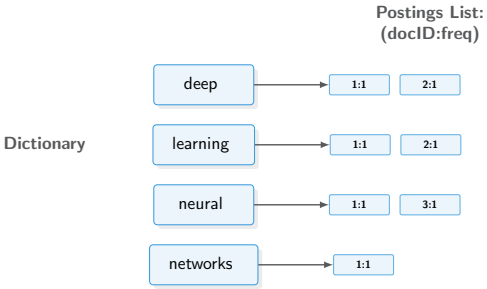
Why still relevant in the age of neural networks?

1. **Exact matching:** Find "iPhone 15 Pro Max" precisely
2. **Speed:** 5-10ms queries on millions of documents
3. **Explainability:** Can see which terms matched
4. **Hybrid search:** Complements dense retrieval

Inverted Index: Core Structure

Documents:

- Doc1: "neural networks for deep learning"
- Doc2: "deep learning applications"
- Doc3: "neural machine translation"

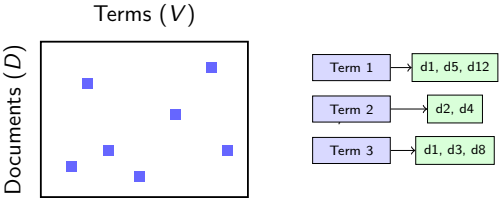


Query: "neural learning"
Process: Lookup "neural" → [1, 3], lookup "learning" → [1, 2]
Intersection: [1] ← Doc 1 contains both terms

Inverted Index as a Sparse Matrix

Vector Space Interpretation

- Documents = Rows (D)
- Terms = Columns (V)
- Values = Weights (TF-IDF, BM25)



Key Insight

The Inverted Index is a **Compressed Sparse Column (CSC)** storage of the Document-Term matrix. We avoid storing zeros!

Why Inverted Index is Fast

Naive Approach:

1. Scan all documents
2. Check if query terms present
3. Rank matches

Complexity: $O(N \cdot M)$

- N = number of documents
- M = avg document length

For 10M docs, 500 words each:
5 **billion** comparisons!

Inverted Index:

1. Lookup term in dictionary
2. Fetch posting list
3. Merge posting lists

Complexity: $O(k \cdot p)$

- k = query terms (typically 2-5)
- p = avg posting list size

For query "neural learning":
Lookup 2 terms, merge 1000 docs
$\rightarrow$ **1000$\times$ faster!**

Posting List Formats: Evolution

Level 1: DocID Only (Boolean Retrieval)

```
neural:  [1, 3, 5, 12, 27, 89]
```

- Supports: AND, OR, NOT queries
- Cannot: Rank by relevance

Level 2: With Frequencies (Ranked Retrieval)

```
neural:  [(1, tf=2), (3, tf=1), (5, tf=3), (12, tf=1), ...]
```

- Supports: TF-IDF, BM25 scoring
- **This is what we use in practice**

Level 3: With Positions (Phrase Queries)

```
neural:  [(1, tf=2, pos=[5,42]), (3, tf=1, pos=[8]), ...]
```

- Supports: "neural networks" (exact phrase)
- Enables: Proximity scoring
- Cost: 2-3× larger index

Score Aggregation: Connecting Indexing and VSM

Vector Space Model Scoring

$$\text{score}(q, d) = \sum_{t \in q \cap d} \text{weight}(t, q) \times \text{weight}(t, d)$$

How it works in the engine:

1. Initialize **Accumulators** (Doc $\rightarrow$ Score)
2. For each term t in query:
3. Traverse postings of t
4. Add contribution to Doc score

Text Preprocessing: Key Decisions

1. Tokenization

Simple: Split on whitespace
Reality: Language-dependent

- English: "don't" → "do" + "n't" or "dont"?
- German: "Informationssystem" → split compound?
- Chinese/Japanese: No spaces between words!

2. Stemming vs. Lemmatization

Stemming (Porter):

- running → run
- university → univers
- Fast, crude

Lemmatization:

- running → run
- better → good
- Accurate, slower

For Neural Models (BERT, embeddings)

Do NOT stem! Pre-trained models expect natural text

Stop Words: Modern Perspective

Traditional View (pre-2010):

- Remove common words: the, a, an, is, are, of, in, ...
- ✔ Smaller index
- ✔ Less noise

Problem:

- "to be or not to be" → ""
- "The Who" (band) → "Who"
- Loses phrase meaning

Modern View (2010+):

- **Keep stop words**
- Storage is cheap
- Important for phrases
- Neural models need them

When to remove:

- Very large scale (web search)
- Memory-constrained devices
- Proven unnecessary for domain

Recommendation

Default: Keep stop words unless you have a specific reason to remove them

Preprocessing Depends on Representation

Sparse Retrieval

- **Tokenization:** Defines vocabulary entries.
- **Stemming:** Useful for term matching (e.g., learn/learning).
- **Stopwords:** Optional (to save space).

Dense Retrieval

- **Tokenizer:** Fixed by the model (BPE, WordPiece).
- **No Stemming:** Models understand word variants.
- **Truncation:** Context window limits matter.

WordPiece Example (BERT):

```
[‘‘un’’, ‘‘##touch’’, ‘‘##able’’]
```

Breaks word into sub-tokens based on statistical frequency.

Index Compression: Why It Matters

The Bottleneck

Modern CPUs: 3+ GHz, billions of operations/second
Modern SSDs: 500 MB/s read speed

Decompression: 2-5 GB/s
Reading uncompressed: 0.5 GB/s

→ **Reading compressed data is faster!**

Real Impact

Collection: 10M documents, 100 words each
Uncompressed index: 10 GB
Compressed (5×): 2 GB

Benefits:

- More index fits in RAM (faster queries)
- Lower storage costs
- Faster I/O (disk → memory)

Gap Encoding: The Key Insight

Original posting list (docIDs):

$$[5, 127, 214, 215, 512, 1024, 2048, 10239]$$

Observation: Most gaps between consecutive docIDs are small

Store gaps instead:

$$[5, 122, 87, 1, 297, 512, 1024, 8191]$$

Why This Helps

Original: Numbers range 5-10,239 $\rightarrow$ need 14 bits each
Gaps: Most numbers small (< 256) $\rightarrow$ need fewer bits

Small numbers compress much better!

Decoding

Sequential only: $\text{docID}_i = \text{docID}_{i-1} + \text{gap}_i$

Variable Byte (VB) Encoding

Core Idea

Use 1 byte for small numbers, multiple bytes for large numbers

- 7 bits for data
- 1 bit (MSB) for continuation: 1 = last byte, 0 = more bytes follow

Number	Binary	VB Encoding
5	101	1_0000101 (1 byte)
122	1111010	1_1111010 (1 byte)
824	1100111000	0_0000110 1_0111000 (2 bytes)

Example compression:

- Gaps: [5, 122, 87, 1] $\rightarrow$ 4 bytes (VB encoded)
- Uncompressed: 16 bytes (4 $\times$ 32-bit integers)
- **Compression ratio: 4$\times$**

VB Encoding: Step-by-Step Example

Encode 824:

1. Binary representation: 1100111000 (10 bits needed)
2. Needs 2 bytes ($10 > 7$)
3. Split into 7-bit chunks from right:
   - Lower 7 bits: 0111000
   - Upper bits: 0000110
4. First byte (higher bits): 0_0000110 (continuation = 0)
5. Second byte (lower bits): 1_0111000 (continuation = 1, last byte)
6. Result: 00000110 10111000

Decode:

1. Read bytes until continuation bit = 1
2. Strip continuation bits: 0000110 0111000
3. Concatenate: 0000110_0111000 = 1100111000 = 824 ✔

Compression in Practice

Typical Results

- VB encoding: 3-4$\times$ compression
- More sophisticated methods (PForDelta): 6-10$\times$ compression
- Dictionary compression (front coding): 30-50

Production Systems

- **Lucene/Elasticsearch:** Uses FOR (Frame of Reference) with SIMD
- **Google:** Custom compression optimized for their hardware
- **Your projects:** VB encoding is good enough (simple, effective)

Key Takeaway

Compression is not optional — it makes systems **faster** and cheaper

PyTerrier Demo: Sparse Indexing

```python
import pyterrier as pt
pt.init()

# Load dataset
dataset = pt.get_dataset('irds:vaswani')

# Index documents (builds inverted index with compression)
indexer = pt.IterDictIndexer("./vaswani_index", overwrite=True)
index_ref = indexer.index(dataset.get_corpus_iter())
print("Indexing complete!")

# Build retriever (uses BM25 scoring)
bm25 = pt.BatchRetrieve(index_ref, wmodel="BM25")

# Search
results = bm25.search("aerodynamic flow over wings")
print(results[['docno', 'score', 'rank']].head())

# Evaluate on full test set
evaluation = pt.Experiment(
    [bm25],
    dataset.get_topics(),
    dataset.get_qrels(),
    eval_metrics=["map", "ndcg_cut_10", "P_10"]
)
print(evaluation)
#    name       map  ndcg_cut_10      P_10
# 0  BM25  0.234567     0.456789  0.345678
```

This is production-ready sparse retrieval!

The Shift to Dense Vectors

Sparse (BM25)

Doc: "car problems"

Representation:

- Dimensions: 30,000 (vocab)
- car: 1
- problems: 1
- All else: 0
- **99.99% zeros**

Limitation:
Misses "automobile issues"

Dense (Neural)

Doc: "car problems"

Representation:

- Dimensions: 768
- [0.23, -0.41, 0.87, . . .]
- All non-zero
- **100% dense**

Advantage:
Matches "automobile issues"
(similar embedding!)

The Problem

Can't use inverted index (no discrete terms)
Need: **Approximate Nearest Neighbor (ANN) search**

The ANN Search Problem

Task

Given:

- Collection of N vectors: $\{\vec{d}_1, \vec{d}_2, \ldots, \vec{d}_N\}$
- Query vector: $\vec{q}$

Find: k vectors most similar to $\vec{q}$ (typically cosine similarity)

Naive Solution: Exact Search

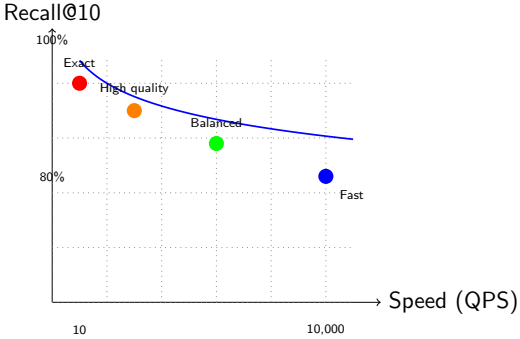
Compare query to **all** N vectors: $O(N \cdot d)$

For $N = 10^9$ docs, $d = 768$ dimensions:
768 billion operations $\rightarrow$ **too slow!** (>1 second even on GPU)

Solution: Approximate Nearest Neighbor (ANN)

Trade exactness for speed: find *approximately* top-k
Typical accuracy: 90-99% recall (good enough for IR!)

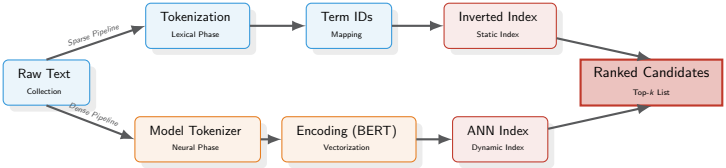
ANN: Accuracy-Speed Trade-off



Typical operating points:

- **High quality:** 95-99% recall @ 100-500 QPS (production RAG)
- **Balanced:** 90-95% recall @ 500-2000 QPS (most use cases)
- **Fast:** 80-90% recall @ 2000-10,000 QPS (first-stage retrieval)

From Text to Index: A Unified View



Key Distinction

Sparse: Matches exact characters (lexical).
Dense: Matches numerical meaning (semantic).

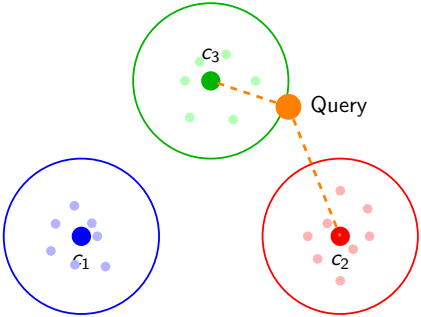
The Standard ANN Approach: IVF + PQ

1. **Clustering-Based (IVF)**
   - Cluster vectors with k-means
   - Search only in nearest clusters
   - ✔ Fast, scalable
   - ✘ Lower accuracy without tuning
2. **Product Quantization (PQ)**
   - Compress vectors aggressively ($32\times$)
   - Approximate distances in compressed space
   - ✔ Very memory-efficient
   - ✘ Moderate accuracy loss
3. **Combined (IVF + PQ)**
   - Use clustering to narrow search area
   - Use compression to fit index in RAM and speed up distance calculation
   - ✔ **The industry standard for large-scale RAG**

IVF: Inverted File Index (for Vectors!)

Core Idea

Cluster vectors, then search only in nearest clusters



Search only in 1-2 nearest clusters (not all!)

IVF: Construction and Search

Construction

```
Run k-means on all vectors → get k cluster centers (e.g., k = 1000)
Assign each vector to its nearest cluster center
Build inverted lists: cluster i → [vector IDs in cluster i]
```

Search

```
Input: Query q⃗, n_probe (clusters to search), k
Compute distance from q⃗ to all cluster centers
Select n_probe nearest cluster centers
candidates ← []
for each selected cluster do
    Fetch all vectors in cluster
    Compute exact distances to q⃗
    Add to candidates
end for
Sort candidates by distance, return top-k
```

Key parameter: n_{probe} (1 = fast/low recall, 100 = slow/high recall)

Product Quantization: Compression

Goal

Compress 768-dim vector (3072 bytes) to 96 bytes (32× compression!)

Method:

1. **Split** each vector into m subvectors (e.g., $m = 96$ subvectors of 8 dims)
2. **Quantize** each subvector: assign to nearest centroid (256 centroids)
3. **Store** centroid ID (1 byte) instead of full subvector

Example

Original: [0.23, -0.41, 0.87, 0.12, -0.34, 0.56, 0.78, -0.23, ...] (768 floats)

Split into 96 subvectors:

- Subvec 1: [0.23, -0.41, 0.87, 0.12, -0.34, 0.56, 0.78, -0.23] → Centroid 42
- Subvec 2: [0.45, 0.67, -0.12, 0.89, 0.34, -0.56, 0.23, 0.11] → Centroid 123
- ... (96 total)

Compressed: [42, 123, 7, 89, ...] (96 bytes)

Product Quantization: Distance Approximation

Key Insight

Can approximate distances using only centroid IDs (no need to decompress!)

Algorithm:

1. **Precompute** distance table: query subvector to all centroids
   - For each of 96 subspaces
   - For each of 256 centroids
   - Total: $96 \times 256 = 24{,}576$ distances (computed once per query)
2. **For each document:**
   - Approximate distance $= \sum_{i=1}^{96} \text{distTable}_i[\text{code}_i]$
   - Just 96 lookups + additions (very fast!)

Performance

- **Memory:** 32× compression (3GB → 96MB for 1M vectors)
- **Speed:** 10-100× faster than exact distance
- **Accuracy:** 80-95% recall typical

IVF + PQ: The Production Standard

Best of Both Worlds

Combine IVF (clustering) with PQ (compression)

Architecture:

1. **Offline:** Cluster vectors into k clusters (IVF)
2. **Offline:** Compress vectors with PQ
3. **Online:** Find n_{probe} nearest clusters
4. **Online:** Search within those clusters using PQ distances

Real-World Impact

Facebook/Meta AI Similarity Search (FAISS):

- 1 billion vectors $\times$ 768 dims $\times$ 4 bytes = **3 TB**
- With IVFPQ: 1 billion vectors $\times$ 64 bytes = **64 GB**
- **50$\times$ memory reduction!**
- Still achieves 90-95% recall with proper tuning

Used by: Meta, OpenAI, many large-scale systems

FAISS Demo: Dense Indexing (IVF+PQ)

```python
import numpy as np
import faiss

# Simulate document embeddings (100k docs, 768 dims)
dimension = 768
n_docs = 100000
doc_embeddings = np.random.randn(n_docs, dimension).astype('float32')
faiss.normalize_L2(doc_embeddings)  # For cosine similarity

# === 1. Training & Building IVF+PQ index ===
n_clusters = 100  # Number of clusters (centroids)
m_subvectors = 96 # Number of PQ subvectors (768/96 = 8 dims each)
bits = 8          # Bits per subvector (256 centroids per subspace)

quantizer = faiss.IndexFlatL2(dimension)
index = faiss.IndexIVFPQ(quantizer, dimension, n_clusters, m_subvectors, bits)

print("Training index...")
index.train(doc_embeddings) # PQ needs training to find centroids
index.add(doc_embeddings)
print(f"Total docs indexed: {index.ntotal}")

# === 2. Search ===
query = np.random.randn(1, dimension).astype('float32')
faiss.normalize_L2(query)

index.nprobe = 10  # Search 10 nearest clusters
distances, indices = index.search(query, k=10)

print("Top 10 document indices:", indices[0])
print("Approximate distances:", distances[0])

# Memory usage (approximation)
# Uncompressed: 100,000 * 768 * 4 bytes = 307.2 MB
# IVFPQ: 100,000 * 96 bytes = 9.6 MB (~32x smaller!)
```

The Decision Matrix: Scale vs. Precision

Aspect	Sparse (Inverted Index)	Dense (IVF+PQ)
Best for	Exact Match, Rare Keywords	Semantic Similarity
Recall	Medium (High for entities)	High (Semantic)
Precision	Very High	High
Latency	5-20ms	10-50ms
Memory	Low	Low-Medium (due to compression)
Scalability	Excellent	Excellent (with IVF)
Explainability	High	Low (Black box)

Production Wisdom

- Use **Sparse** when you need to find exact names, model numbers, or unique terms.
- Use **Dense** when users ask questions in natural language or use synonyms.

Case Study: Academic Paper Search (100M+)

Scenario

Research search engine with 100M papers (arXiv, PubMed, etc.)

The Challenge:

- Cannot hold 100M raw vectors in RAM (300GB+).
- Users need both exact titles (Sparse) and broad conceptual search (Dense).

The Strategy: IVF+PQ

- **Sparse (BM25):** Index title, authors, abstracts for keyword matching.
- **Dense (IVF+PQ):**
  - Cluster into 10k centroids (high speed).
  - Compress vectors 32× (fits on a single server!).

Result: Billion-scale search with low latency and low cost.

Common Pitfalls and Solutions

Pitfall 1: Using Only Dense for Entity-Heavy Domains

Problem: Query "Python 3.11 release date" $\rightarrow$ matches "snake release from zoo"
Solution: Use hybrid! BM25 catches "Python 3.11" exactly

Pitfall 2: Not Tuning ANN Parameters

Problem: Using default `ef_search=16` $\rightarrow$ only 80% recall
Solution: Tune on dev set. Start `ef_search=64`, increase until recall plateaus

Pitfall 3: Inconsistent Preprocessing

Problem: Index with stemming, query without $\rightarrow$ no matches!
Solution: Same preprocessing for both. For embeddings: no stemming!

Pitfall 4: Forgetting to Normalize Vectors

Problem: Using unnormalized embeddings $\rightarrow$ Euclidean distance $\neq$ cosine similarity
Solution: Always normalize: `faiss.normalize_L2(embeddings)`

Production Checklist

Before deploying, verify:

Sparse Index:

- ☐ Compression enabled
- ☐ Stop words handled
- ☐ Stemming appropriate
- ☐ Field boosting configured
- ☐ Metadata filterable

Dense Index (IVF+PQ):

- ☐ Vectors normalized
- ☐ Number of probes (n_probe) tuned
- ☐ Memory budget met
- ☐ PQ training data is representative

Monitoring:

- ☐ Track recall@k (vs. exact search)
- ☐ Monitor p95/p99 latency
- ☐ Alert on index growth
- ☐ Log query/document drift

Modern IR Infrastructure Examples

Production RAG (Cloud/Managed)

- **Embedding:** OpenAI `text-embedding-3-small`
- **Vector DB:** Pinecone, Qdrant, or Milvus Managed.
- **Candidate Generation:** Dense-first (IVF or HNSW hidden by provider).
- **LLM:** GPT-4o / Claude 3.5 Sonnet.

Large Scale Open Source

- **Sparse:** Elasticsearch / OpenSearch (self-hosted).
- **Dense:** FAISS (IVF+PQ) for billionaire-scale efficiency.
- **Language:** Python / C++ for performance.
- **Next Step:** Re-ranking with Cross-encoders (Lecture 4).

What We Covered Today

1. **Sparse Indexing (Classical)**
   - Inverted index structure
   - Text preprocessing pipeline
   - Compression (Gap encoding + VB)
   - PyTerrier demo
2. **Dense Indexing (Neural)**
   - Why ANN search is needed
   - IVF+PQ: The standard for scale (Clustering + Compression)
   - FAISS demo
3. **Design Decisions**
   - Modern IR Pipeline (Candidate Generation)
   - Traditional IR vs. RAG
   - Scaling to billions of documents

Key Takeaways

1. Inverted Indexes are the Backbone

Keyword matching is essential for exact queries and entities. Don't throw away BM25!

2. Dense Retrieval Enables Semantic Search

Embeddings capture meaning, but require ANN techniques like IVF+PQ to work at scale.

3. Candidate Generation is the Bottleneck

Fast indexing determines how many documents your re-rankers or LLMs can "see".

4. Scalability requires Compression

Techniques like Product Quantization (PQ) are vital for fitting large indexes in RAM.

Connection to Next Lectures

Lecture 3: Ranking Models

- BM25 scoring uses inverted indexes we built today
- Query expansion (RM3) operates on sparse retrieval
- Language models for better ranking

Lecture 4: Transformer Re-rankers

- Re-rank candidates from hybrid retrieval
- Cross-encoders (BERT) for precision
- Two-stage: retrieve (today) $\rightarrow$ re-rank (lecture 4)

Lecture 5: Dense Retrieval (Deep Dive)

- Bi-encoders in detail
- Training dense retrievers
- We already know ANN indexing!

Lecture 6: Learning to Rank

- Combine sparse + dense scores optimally
- Learn fusion weights from data
- Put it all together

Assignment 2: From Sparse to Dense Retrieval

Due: 2 weeks

Tasks:

1. **Sparse Indexing (40 pts)**
   - Build inverted index with PyTerrier.
   - Measure effect of compression (VB) on index size.
2. **Dense Indexing (40 pts)**
   - Build IVF+PQ index with FAISS on 100k vectors.
   - Measure recall vs. exact search as you change `n_probe`.
3. **Analysis (20 pts)**
   - Compare latency and memory usage of Sparse vs. Dense.
   - Identify query types where each method fails.

Resources

Documentation

- PyTerrier: `https://pyterrier.readthedocs.io`
- FAISS: `https://github.com/facebookresearch/faiss/wiki`
- Sentence Transformers: `https://www.sbert.net`

Papers (Optional Reading)

- HNSW: Malkov & Yashunin (2018) - "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs"
- Product Quantization: Jégou et al. (2011) - "Product quantization for nearest neighbor search"
- Hybrid Search: Lin et al. (2021) - "A Few Brief Notes on DeepImpact, COIL, and a Conceptual Framework for Information Retrieval Techniques"

Tools

- Vector databases: Pinecone, Weaviate, Qdrant, Milvus
- Elasticsearch 8.0+ (native kNN + BM25)
- Vespa (Yahoo's search engine, open source)

Questions?

Questions?

Office Hours: [Your time]

Email: [Your email]

Next Class: Lecture 3 — Ranking Models

We'll use the indexes we built today to implement BM25, language models, and query expansion!

Start the assignment early — building a hybrid system takes time!